PYTHON PROGRAMMING

Mini Project Assignment Book

Assignment Overview

Assignment Title	Python Mini Project Assignment Book
Total Projects	5 (Easy to Moderate)
Submission Mode	Individual
Language	Python 3.x (No external libraries)
Tools Allowed	VS Code, PyCharm, IDLE, Jupyter Notebook
Total Marks	70 marks per project (+ 10 Bonus)

Objective

This assignment is designed to develop practical programming skills in Python by building real-world mini projects using only Python fundamentals. Each project covers multiple core concepts and is structured to simulate actual industry use cases at a beginner-to-intermediate level.

Topics Covered

- Variables and Data Types
- Input and Output Operations
- Arithmetic, Comparison, and Logical Operators
- Conditional Statements (if, if-else, nested if, elif)
- Loops (for, while, break, continue)
- Functions (with parameters, return values)
- Lists, Tuples, Sets, and Dictionaries
- String Operations and Formatting
- Basic File Handling (read, write, append)
- Exception Handling (try, except, finally)

Project Summary Table

#	Project Title	Difficulty	Est. Time	Key Concepts
1	Student Management System	Easy	3-5 Hours	Dict, List, Functions
2	Library Management System	Easy-Mod	4-6 Hours	Nested Dict, Loops
3	Personal Expense Tracker	Moderate	5-7 Hours	Dict, Sets, Arithmetic
4	Quiz & Examination System	Moderate	5-7 Hours	Tuples, String Ops
5	Inventory Management System	Mod-Adv	6-8 Hours	File Handling, Sets

General Submission Rules

1. Each project must be submitted as a separate .py file.
2. File names must follow the convention provided in each project.

3. All code must include inline comments explaining key sections.
4. Programs must run without errors in Python 3.x.
5. Copied code will receive zero marks. All work must be original.
6. Late submissions will incur a 5-mark deduction per day.

Project 1: Student Management System

Project 1: Student Management System

Difficulty Level: Easy

Project Title	Student Management System
Difficulty	Easy
Estimated Time	3-5 Hours

1. Project Description

The Student Management System is a console-based Python application that allows administrators to manage student records efficiently. Users can add, view, update, search, and delete student information such as name, roll number, marks, and grade. This project forms the foundation of data management and real-world CRUD (Create, Read, Update, Delete) operations using Python.

2. Real-World Use Case

Schools and colleges maintain large databases of students. This project simulates the basic backend logic used in education management portals like university dashboards, school ERPs, and examination management software.

3. Learning Objectives

- Understand how to use dictionaries and lists to store structured data.
- Practice taking user input and displaying formatted output.
- Implement conditional logic to validate data.
- Use loops to iterate through records and perform operations.
- Apply functions for modular and reusable code design.

4. Python Concepts Used

- Variables and Data Types (int, str, float)
- Input / Output (input(), print())
- Lists and Dictionaries
- Conditional Statements (if, if-else)
- Loops (for, while)
- Functions
- String Operations
- Exception Handling (try-except for invalid inputs)

5. Input Requirements

- Student Name (string)
- Roll Number (integer, unique)
- Marks in 5 Subjects (float each)
- User menu choice (integer 1-6)

6. Output Requirements

- Formatted student record display

- Calculated percentage and grade
- Success/failure messages for operations
- Search results with complete student profile

7. Step-by-Step Algorithm

7. Initialize an empty dictionary to store student records.
8. Display a menu: Add / View All / Search / Update / Delete / Exit.
9. Accept user menu choice with input validation.
10. For Add: take name, roll number, and 5 subject marks. Calculate percentage and grade. Store in dictionary with roll number as key.
11. For View: iterate through dictionary and print all records in formatted table.
12. For Search: prompt roll number, look up dictionary, print record if found.
13. For Update: find record by roll number, allow field-by-field update.
14. For Delete: confirm deletion, then remove entry from dictionary.
15. Repeat menu until user selects Exit.

8. Flow of Execution

START → Initialize data structure → Show Menu → Accept Choice → Execute Operation → Show Result → Return to Menu → EXIT on choice 6

9. Sample Input / Output

```
====== STUDENT MANAGEMENT SYSTEM ======
1. Add Student    2. View All
3. Search         4. Update
5. Delete         6. Exit
Enter choice: 1

Enter Name       : Rahul Sharma
Enter Roll No    : 101
Marks (Sub 1-5)  : 78 85 90 72 88

=== Record Added Successfully ===
Name: Rahul Sharma | Roll: 101 | %: 82.60 | Grade: B+
```

10. Project Modules / Functions to Create

- add_student() — Collects and validates student data
- view_all() — Displays all records in formatted output
- search_student() — Finds a student by roll number
- update_student() — Updates specific fields of a record
- delete_student() — Removes a student entry
- calculate_grade() — Returns grade based on percentage
- show_menu() — Displays menu and returns user choice

11. Expected Features

- Duplicate roll number prevention
- Automatic percentage and grade calculation
- Input validation using try-except
- Clean formatted output for all records

12. Additional Challenges (Advanced Students)

- Save records to a .txt or .csv file and load on startup
- Sort students by percentage in descending order
- Generate class report: topper, average %, pass/fail count
- Add rank field based on marks comparison

13. Evaluation Criteria

Criteria	Marks	Description
Correct CRUD Operations	**20**	All operations work without errors
Input Validation & Error Handling	**15**	Invalid inputs handled gracefully
Code Modularity (Functions)	**15**	Proper use of functions
Output Formatting	**10**	Clean, readable output
Additional Challenges	**10**	Bonus for extras

14. Submission Guidelines

- File name: student_management.py
- Include comments explaining each function
- Submit via college portal or as email attachment
- Deadline: As announced by the instructor

Project 2: Library Management System

Project 2: Library Management System

Difficulty Level: Easy–Moderate

Project Title	Library Management System
Difficulty	Easy to Moderate
Estimated Time	4-6 Hours

1. Project Description

The Library Management System enables librarians and students to manage book records within a library. The system tracks books (title, author, ISBN, availability), handles book issue and return operations, tracks borrowers, and generates availability reports. This project introduces the concept of nested data structures and state tracking.

2. Real-World Use Case

Public libraries, college libraries, and digital book repositories all rely on systems similar to this. Platforms such as KOHA (open-source library software) and institutional library portals use similar CRUD and availability-tracking logic at their core.

3. Learning Objectives

- Work with nested dictionaries (book record within a master dictionary)
- Track state changes (Available / Issued) using boolean or string flags
- Design menu-driven programs using while loops
- Implement search by multiple fields (title, author, ISBN)
- Apply list operations to manage borrower queues

4. Python Concepts Used

- Dictionaries (nested) and Lists
- Conditional Statements (if-elif-else)
- While Loops with menu logic
- Functions for modular design
- String Operations (upper(), strip(), find())
- Exception Handling (KeyError, ValueError)

5. Input Requirements

- Book Title (string)
- Author Name (string)
- ISBN Number (string, unique identifier)
- Borrower Name and Student ID (for issue/return)
- Menu choice (integer)

6. Output Requirements

- Full book catalog with availability status
- Issue confirmation with due date (7 days from issue)
- Return confirmation and updated status

- Search results with all book details

7. Step-by-Step Algorithm

16. Create a master dictionary: ISBN → {title, author, available, borrower, issue_date}.
17. Show main menu: Add Book / Issue Book / Return Book / Search / View Catalog / Exit.
18. Add Book: collect ISBN, title, author. Default available=True, borrower=None.
19. Issue Book: search ISBN, check availability. If available, set available=False, record borrower and date.
20. Return Book: search ISBN, check if issued. Set available=True, clear borrower info.
21. Search: accept keyword, scan all books for title/author match, display results.
22. View Catalog: iterate and print all books with formatted columns.
23. Repeat until Exit.

8. Flow of Execution

START → Load book data → Show Menu → Operation → Validate → Update State → Display Result → Loop → EXIT

9. Sample Input / Output

```
====== LIBRARY MANAGEMENT SYSTEM ======
1. Add Book    2. Issue Book
3. Return Book 4. Search Book
5. View All    6. Exit
Enter choice: 2

Enter ISBN        : 978-3-16-148410-0
Enter Borrower ID : STU2024-07
Enter Your Name   : Priya Desai

Book Issued Successfully!
Title    : Python Programming
Due Date: 28-June-2026
```

10. Modules / Functions to Create

- add_book() — Register a new book
- issue_book() — Issue a book to a student
- return_book() — Process book return
- search_book() — Search by title or author
- view_catalog() — Display full library catalog
- check_due_date() — Calculate due date from issue date

11. Expected Features

- Unique ISBN enforcement during registration
- Availability toggle (Issued / Available)
- Borrower tracking with issue date
- Multi-field search (title and author)

12. Additional Challenges

- Maintain a waiting list for already-issued books
- Track overdue books and calculate fine (Rs. 2/day after 7 days)
- Export catalog to a text file

- Count total books issued vs. available

13. Evaluation Criteria

Criteria	Marks	Description
Book CRUD Operations	**20**	Add, View, Search work correctly
Issue & Return Logic	**20**	State transitions are accurate
Exception Handling	**10**	Invalid ISBN, already issued handled
Code Clarity & Comments	**10**	Well-structured, readable code
Bonus Features	**10**	Fine calculation or file export

14. Submission Guidelines

- File name: library_management.py
- Demonstrate at least 5 books in the catalog
- Test issue → return cycle for at least 2 books
- Submit before deadline with sample test output as a comment

Project 3: Personal Expense Tracker

Project 3: Personal Expense Tracker

Difficulty Level: Moderate

Project Title	Personal Expense Tracker
Difficulty	Moderate
Estimated Time	5-7 Hours

1. Project Description

The Personal Expense Tracker is a Python console application that helps users track their daily expenses by category (Food, Travel, Entertainment, etc.), calculate monthly totals, compare expenses against a set budget, and identify overspending. The application uses lists of dictionaries to maintain a running expense log and applies string formatting for clear reporting.

2. Real-World Use Case

Financial apps like Walnut, Money Manager, and YNAB (You Need A Budget) operate on similar principles. Even simple spreadsheet-based expense tools in corporate environments use the same category-based tracking logic that this project implements.

3. Learning Objectives

- Store structured records using a list of dictionaries
- Apply arithmetic operators for financial calculations
- Use string formatting for clean financial reports
- Filter list data using conditions inside loops
- Implement budget comparison logic with conditional statements

4. Python Concepts Used

- Lists of Dictionaries
- Arithmetic and Comparison Operators
- Conditional Statements (if, if-else, nested if)
- For Loops (for filtering and summation)
- Functions (modular expense operations)
- String Operations and Formatting (f-strings)
- Sets (to derive unique expense categories)
- Exception Handling (invalid amounts)

5. Input Requirements

- Expense Description (string)
- Category (string: Food / Travel / Bills / Entertainment / Other)
- Amount (float, positive)
- Date of expense (string in DD-MM-YYYY format)
- Monthly Budget limit (float)

6. Output Requirements

- Itemized expense list with serial numbers

- Category-wise expense summary
- Total spending vs. budget comparison
- Warning message if budget exceeds 80% or 100%
- Top spending category identification

7. Step-by-Step Algorithm

24. Initialize an empty list for expenses and prompt user to set monthly budget.
25. Show menu: Add Expense / View All / Category Summary / Budget Report / Exit.
26. Add Expense: collect description, category, amount, date. Validate amount > 0. Append dict to list.
27. View All: iterate and print all expenses with index, description, category, amount, date.
28. Category Summary: use a dictionary to accumulate totals per category. Display results.
29. Budget Report: sum all amounts. Compare with budget. Display percentage used. Show warning if needed.
30. Loop until Exit.

8. Flow of Execution

START → Set Budget → Menu → Add / View / Summarize / Report → Validate → Calculate → Display → Loop → EXIT

9. Sample Input / Output

```
===== PERSONAL EXPENSE TRACKER =====
Monthly Budget: Rs. 5000.00

1.Add  2.View  3.Category  4.Report  5.Exit
Choice: 4

========= BUDGET REPORT =========
Total Spent    : Rs. 4200.00
Budget Limit   : Rs. 5000.00
Remaining      : Rs.  800.00
Used           : 84.00%

⚠ WARNING: You have used 84% of your budget!
Top Category   : Food (Rs. 1800.00)
```

10. Modules / Functions to Create

- add_expense() — Collects and validates expense entry
- view_expenses() — Prints all entries in table format
- category_summary() — Groups and totals by category
- budget_report() — Compares spending against budget
- get_top_category() — Returns highest spending category
- validate_amount() — Ensures positive float input

11. Expected Features

- Budget set at session start with persistence during run
- Category-wise breakdown using dictionary aggregation
- Color-coded warning levels (text labels for console)
- Running total visible after each new entry

12. Additional Challenges

- Filter and display expenses by a specific category
- Find the most expensive single expense
- Calculate average daily spending
- Save expense log to expenses.txt at exit

13. Evaluation Criteria

Criteria	Marks	Description
Expense Addition & Validation	**15**	Correct data entry with validation
Category Summary Logic	**20**	Accurate grouping and totals
Budget Report Accuracy	**20**	Correct calculations and warnings
Code Structure & Functions	**15**	Modular, well-commented code
Bonus Features	**10**	File save or filter by category

14. Submission Guidelines

- File name: expense_tracker.py
- Include at least 10 sample expense entries in a test run
- Demonstrate budget warning functionality
- Submit with a screenshot or copy of sample output

Project 4: Quiz and Examination System

Project 4: Quiz and Examination System

Difficulty Level: Moderate

Project Title	Quiz and Examination System
Difficulty	Moderate
Estimated Time	5-7 Hours

1. Project Description

The Quiz and Examination System is an interactive Python application that presents multiple-choice questions (MCQs) to a student, evaluates responses, tracks scores, and generates a detailed result report at the end. The question bank is stored using tuples (immutable, for data integrity) and the application handles timing, scoring, and grade calculation automatically.

2. Real-World Use Case

Online examination platforms such as TestBook, BYJU's Exam Prep, and university LMS (Learning Management Systems) like Moodle or Google Classroom quizzes are built on similar foundations of storing questions, accepting answers, evaluating correctness, and producing result reports.

3. Learning Objectives

- Use tuples to store immutable question data
- Implement MCQ logic with 4 options and 1 correct answer
- Calculate score, percentage, and grade automatically
- Apply string operations for input normalization (upper/lower/strip)
- Use randomization to shuffle questions (random module)

4. Python Concepts Used

- Tuples (for storing question bank — immutable)
- Lists (for tracking answers and scores)
- Dictionaries (for storing student results)
- Conditional Statements (answer checking, grading)
- For Loops (question iteration)
- Functions (question display, scoring, reporting)
- String Operations (strip, upper for normalization)
- Sets (to ensure no duplicate questions are repeated)
- Exception Handling (invalid option input)

5. Input Requirements

- Student Name (string)
- Student Roll Number (integer)
- Answer to each question (A, B, C, or D)

6. Output Requirements

- Question display with 4 options (A, B, C, D)

- Correct / Incorrect feedback after each answer
- Final score: X out of total
- Percentage and grade in result report
- List of wrong answers with correct answers shown

7. Step-by-Step Algorithm

31. Define question bank as list of tuples: (question, optA, optB, optC, optD, correct_option).
32. Collect student name and roll number.
33. Initialize score = 0 and wrong_answers = [].
34. Loop through each question: display question and options, accept and validate answer.
35. If answer matches correct option, increment score. Else, append to wrong_answers list.
36. After all questions, calculate percentage = (score / total) * 100.
37. Assign grade based on percentage range.
38. Display detailed result report.

8. Flow of Execution

START → Student Login → Load Questions → For each question: Show → Accept → Validate → Score → Show Result Report → EXIT

9. Sample Input / Output

```
===== PYTHON QUIZ SYSTEM =====
Student: Anjali Patil | Roll: 204

Q1: Which data type is IMMUTABLE in Python?
   A. List      B. Dictionary
   C. Tuple     D. Set
Your Answer: C
✓ Correct!

====== RESULT REPORT ======
Name    : Anjali Patil
Score   : 8 / 10
Percent : 80.00%
Grade   : B+
Result  : PASS
```

10. Modules / Functions to Create

- load_questions() — Returns the question bank (list of tuples)
- display_question() — Displays a question with options
- get_answer() — Accepts and validates student answer
- evaluate_quiz() — Scores all answers and compiles result
- calculate_grade() — Returns grade from percentage
- show_report() — Prints final formatted result report
- show_wrong_answers() — Displays incorrectly answered questions

11. Expected Features

- Minimum 10 questions in the question bank
- Input validation — only A, B, C, D accepted
- Score tracking throughout the quiz
- Final report with wrong-answer review

- Pass/Fail verdict based on 50% threshold

12. Additional Challenges

- Shuffle questions randomly using random.shuffle()
- Add a timer per question (10 seconds) using time module
- Allow category-based quiz (Python Basics / OOP / File Handling)
- Store high scores in a file and display leaderboard

13. Evaluation Criteria

Criteria	Marks	Description
Question Bank Design (Tuples)	**15**	Proper use of tuple structure
Answer Evaluation Logic	**20**	Correct scoring and feedback
Grade Calculation	**15**	Accurate percentage and grade
Wrong Answer Review	**10**	Missed answers shown at end
Bonus Features	**10**	Timer or shuffle or leaderboard

14. Submission Guidelines

- File name: quiz_system.py
- Include at least 10 unique questions in the question bank
- Test and document: full quiz run in sample output comments
- Bonus: include at least one additional challenge feature

Project 5: Inventory Management System

Project 5: Inventory Management System

Difficulty Level: Moderate–Advanced

Project Title	Inventory Management System
Difficulty	Moderate to Advanced
Estimated Time	6-8 Hours

1. Project Description

The Inventory Management System is a comprehensive stock-control application built in Python. It tracks products by ID, name, category, price, and quantity. The system allows stock-in (restocking), stock-out (sales), generates low-stock alerts, and produces inventory reports with total stock value. It uses all core Python data structures and integrates basic file handling for persistent storage.

2. Real-World Use Case

Retail stores, warehouses, pharmacies, and e-commerce backends all depend on inventory systems. Solutions like Zoho Inventory, QuickBooks, and Tally ERP implement the same principles of stock tracking, reorder alerts, and valuation reports that this project demonstrates at a foundational level.

3. Learning Objectives

- Integrate multiple Python data structures in a single application
- Implement stock-in and stock-out transaction logic
- Generate aggregate reports using loops and arithmetic
- Apply sets to manage unique product categories
- Use file handling to save and load inventory data

4. Python Concepts Used

- Dictionaries (product ID → product dict)
- Lists (transaction history log)
- Sets (unique product categories)
- Tuples (for fixed product metadata snapshots)
- Conditional Statements (low-stock alert, out-of-stock check)
- For / While Loops
- Functions (modular operations)
- String Operations (strip, upper, format)
- Basic File Handling (read/write to .txt)
- Exception Handling (FileNotFoundError, ValueError)

5. Input Requirements

- Product ID (string, unique: P001, P002...)
- Product Name (string)
- Category (string)
- Unit Price (float)

- Quantity (integer)
- Reorder Level (integer — minimum before alert)
- Stock-in / Stock-out quantity (integer)

6. Output Requirements

- Full inventory list with all product details
- Low-stock alert list (products below reorder level)
- Total inventory value (sum of price × quantity for all items)
- Transaction log for all stock movements
- Category-wise stock summary

7. Step-by-Step Algorithm

39. Load inventory from file (inventory.txt) if it exists; else start with empty dict.
40. Show menu: Add Product / Stock In / Stock Out / View Inventory / Low Stock Alert / Report / Save & Exit.
41. Add Product: collect all fields, validate unique product ID, add to inventory dict.
42. Stock In: find product by ID, add quantity, append transaction to log.
43. Stock Out: find product, check sufficient stock. If yes, deduct. If no, show error.
44. View Inventory: print all records in formatted columns including calculated total value.
45. Low Stock Alert: iterate inventory, list all products where qty <= reorder_level.
46. Report: total products, total value, categories (from set), top product by value.
47. Save & Exit: write inventory dict to file. Exit.

8. Flow of Execution

START → Load File → Menu → Validate → Execute Operation → Log Transaction → Update Dict → Display → Loop → Save → EXIT

9. Sample Input / Output

```
===== INVENTORY MANAGEMENT SYSTEM =====
Products Loaded: 12 | Categories: 4

1.Add  2.Stock-In  3.Stock-Out
4.View 5.Alert     6.Report  7.Exit
Choice: 6

======= INVENTORY REPORT =======
Total Products    : 12
Total Stock Value : Rs. 1,24,500.00
Categories        : Electronics, FMCG, Apparel, Stationery
Low Stock Items   : 3 (below reorder level)
Highest Value Item: Laptop (Rs. 58,000 x 2 = Rs. 1,16,000)
```

10. Modules / Functions to Create

- load_inventory() — Read data from inventory.txt
- save_inventory() — Write data to inventory.txt
- add_product() — Register new product
- stock_in() — Add quantity to existing product
- stock_out() — Deduct quantity with validation
- view_inventory() — Display full formatted table

- low_stock_alert() — List products below reorder level
- generate_report() — Summary statistics and analytics
- log_transaction() — Append entry to transaction history
- get_total_value() — Returns total inventory valuation

11. Expected Features

- Unique Product ID enforcement
- Stock level validation before stock-out
- Automatic low-stock detection at reorder level
- Transaction log with timestamp and type (IN/OUT)
- File persistence: save at exit, load at startup

12. Additional Challenges

- Generate daily transaction report filtered by date
- Implement product search by name or category
- Add GST calculation (5%, 12%, or 18%) based on category
- Create a simple barcode-style Product ID generator
- Build a restock suggestion list sorted by urgency

13. Evaluation Criteria

Criteria	Marks	Description
Product CRUD & Validation	**15**	Add, view with proper validation
Stock-In / Stock-Out Logic	**20**	Accurate transactions, no negatives
Low Stock Alert System	**15**	Correct alert generation
Report Generation	**15**	Accurate totals and summaries
File Handling (Load/Save)	**10**	Data persists across sessions
Bonus Features	**10**	Extra features as listed above

14. Submission Guidelines

- File names: inventory_management.py and inventory.txt
- Pre-populate at least 8 products across 3 categories
- Demonstrate stock-in, stock-out, and report in one test run
- Submit both .py file and sample inventory.txt

General Grading Rubric

The following rubric applies to all five projects and outlines how marks are distributed for standard evaluation:

Evaluation Parameter	Max Marks	Description
Functionality & Correctness	**25**	All required features work correctly
Code Structure & Modularity	**15**	Proper use of functions, clean layout
Input Validation & Error Handling	**10**	Handles invalid inputs, exceptions caught
Output Formatting	**10**	Clear, labeled, readable output
Comments & Documentation	**10**	Inline comments and function docstrings
Bonus / Additional Challenges	**10**	Any extra feature as listed in project

Python Quick Reference

Data Structures at a Glance

Structure	Syntax	Mutable?	Use Case
List	[ ]	Yes	Ordered, changeable collection
Tuple	( )	No	Fixed data (question banks)
Set	{ }	Yes	Unique values (categories)
Dictionary	{ key: val }	Yes	Key-value record storage

Grade Scale Reference

Grade	Percentage	Description	Status
O	90-100%	Outstanding	PASS
A+	80-89%	Excellent	PASS
A	70-79%	Very Good	PASS
B+	60-69%	Good	PASS
B	50-59%	Average	PASS
F	< 50%	Fail	FAIL

Appendix — Tips for Students

A. Recommended Development Approach

48. Read the complete project description before writing a single line of code.
49. Sketch the data structures (what variables, what types?) on paper first.
50. Write functions one at a time, test each before moving to the next.
51. Use meaningful variable names (student_name, not sn or x).
52. Run the program after every 10-15 lines to catch errors early.

B. Common Python Errors to Avoid

- IndentationError — Python relies on indentation; be consistent with 4 spaces.
- KeyError — Always check if a key exists in a dict before accessing it.
- TypeError — Ensure correct types (int vs str) before arithmetic operations.
- ZeroDivisionError — Guard against division by zero in calculations.
- IndexError — Check list length before accessing by index.

C. File Naming Convention

- student_management.py — Project 1
- library_management.py — Project 2
- expense_tracker.py — Project 3
- quiz_system.py — Project 4
- inventory_management.py — Project 5

D. Academic Integrity

All submitted work must be your own. Discussing concepts with classmates is encouraged, but sharing code or copying from the internet is strictly prohibited. Any submission found to be plagiarized will receive a zero and may be subject to disciplinary action as per university policy.

--- End of Assignment Book ---

Best of luck with your Python journey!